



Threagile

Agile Threat Modeling

Threat Model Report

VaultNote Threat Model

18 April 2026

VaultNote Security Team

Table of Contents

Results Overview

Management Summary	4
Impact Analysis of 11 Initial Risks in 4 Categories	5
Risk Mitigation	6
Asset Register	7
Impact Analysis of 11 Remaining Risks in 4 Categories	10
Application Overview	11
Data-Flow Diagram	12
Security Requirements	13
Abuse Cases	14
Tag Listing	15
STRIDE Classification of Identified Risks	17
Assignment by Function	19
RAA Analysis	20
Data Mapping	21
Out-of-Scope Assets: 0 Assets	22
Potential Model Failures: 0 / 0 Risks	23
Questions: 0 / 0 Questions	24

Risks by Vulnerability category

Identified Risks by Vulnerability category	25
Application Threat - Strictly-Confidential Asset Bridges Business Process Contexts: 2 / 26 Risks	26
Operational Threat - Critical Business Process Supported by Non-Redundant Assets: 3 / 28 Risks	28
Operational Threat - Critical Business Process Without Operational Monitoring: 3 / 3 Risks	30
Operational Threat - Critical Business Process on Shared Runtime: 3 / 3 Risks	32

Risks by Technical Asset

Identified Risks by Technical Asset	35
API Server: 4 / 4 Risks	36
PostgreSQL Database: 4 / 4 Risks	40
Redis Cache: 3 / 3 Risks	43
Browser SPA: 0 / 0 Risks	45
MinIO Object Storage: 0 / 0 Risks	47
Nginx Reverse Proxy: 0 / 0 Risks	49

Data Breach Probabilities by Data Asset

Identified Data Breach Probabilities by Data Asset	51
File Attachments: 4 / 4 Risks	52

Note Content: 8 / 8 Risks	53
Session Tokens: 7 / 7 Risks	54
User Credentials: 8 / 8 Risks	55

Trust Boundaries

Application Network	56
Data Tier	56
DMZ	56
Internet	56

Shared Runtime

Docker Host	58
-------------	----

About Threagile

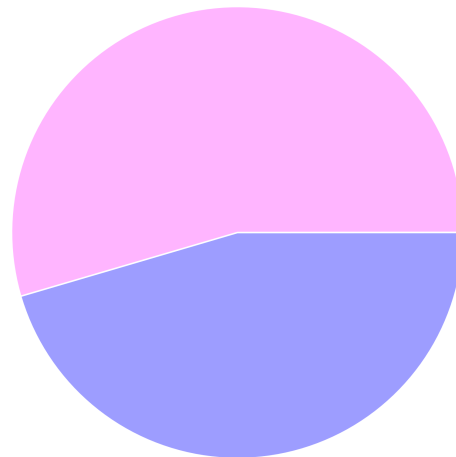
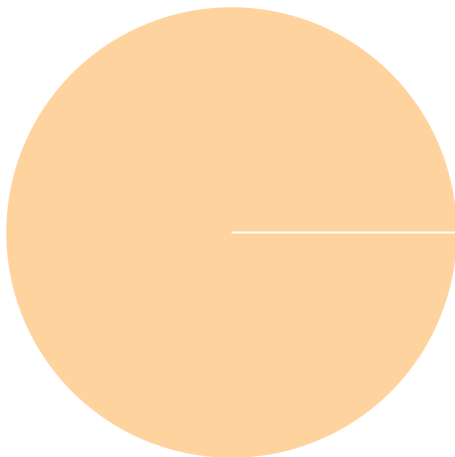
Risk Rules Checked by Threagile	59
Disclaimer	60

Management Summary

Threagile toolkit was used to model the architecture of "VaultNote Threat Model" and derive risks by analyzing the components and data flows. The risks identified during this analysis are shown in the following chapters. Identified risks during threat modeling do not necessarily mean that the vulnerability associated with this risk actually exists: it is more to be seen as a list of potential risks and threats, which should be individually reviewed and reduced by removing false positives. For the remaining risks it should be checked in the design and implementation of "VaultNote Threat Model" whether the mitigation advices have been applied or not.

Each risk finding references a chapter of the OWASP ASVS (Application Security Verification Standard) audit checklist. The OWASP ASVS checklist should be considered as an inspiration by architects and developers to further harden the application in a Defense-in-Depth approach. Additionally, for each risk finding a link towards a matching OWASP Cheat Sheet or similar with technical details about how to implement a mitigation is given.

In total **11 initial risks** in **4 categories** have been identified during the threat modeling process:



VaultNote is a containerized note-taking application comprised of a React SPA, Nginx reverse proxy, Node.js/Express REST API, PostgreSQL database, Redis session store, and MinIO object storage. The architecture deliberately contains several intentional misconfigurations (unencrypted internal HTTP, no encryption at rest, hardcoded credentials) to demonstrate automated threat detection via Threagile. All findings below are expected and serve as demonstration targets.

Impact Analysis of 11 Initial Risks in 4 Categories

The most prevalent impacts of the **11 initial risks** (distributed over **4 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated: **Application Threat - Strictly-Confidential Asset Bridges Business Process Contexts: 2 Initial Risks** - Exploitation likelihood is *Likely* with *High* impact.

Credential exposure in any one process context compromises all other processes the asset supports, amplifying the breach blast radius.

Elevated: **Operational Threat - Critical Business Process Supported by Non-Redundant Assets: 3 Initial Risks** - Exploitation likelihood is *Likely* with *High* impact.

Complete outage of a critical business process due to failure or targeted attack against the sole supporting technical asset. No failover path means maximum downtime.

Elevated: **Operational Threat - Critical Business Process Without Operational Monitoring: 3 Initial Risks** - Exploitation likelihood is *Likely* with *High* impact.

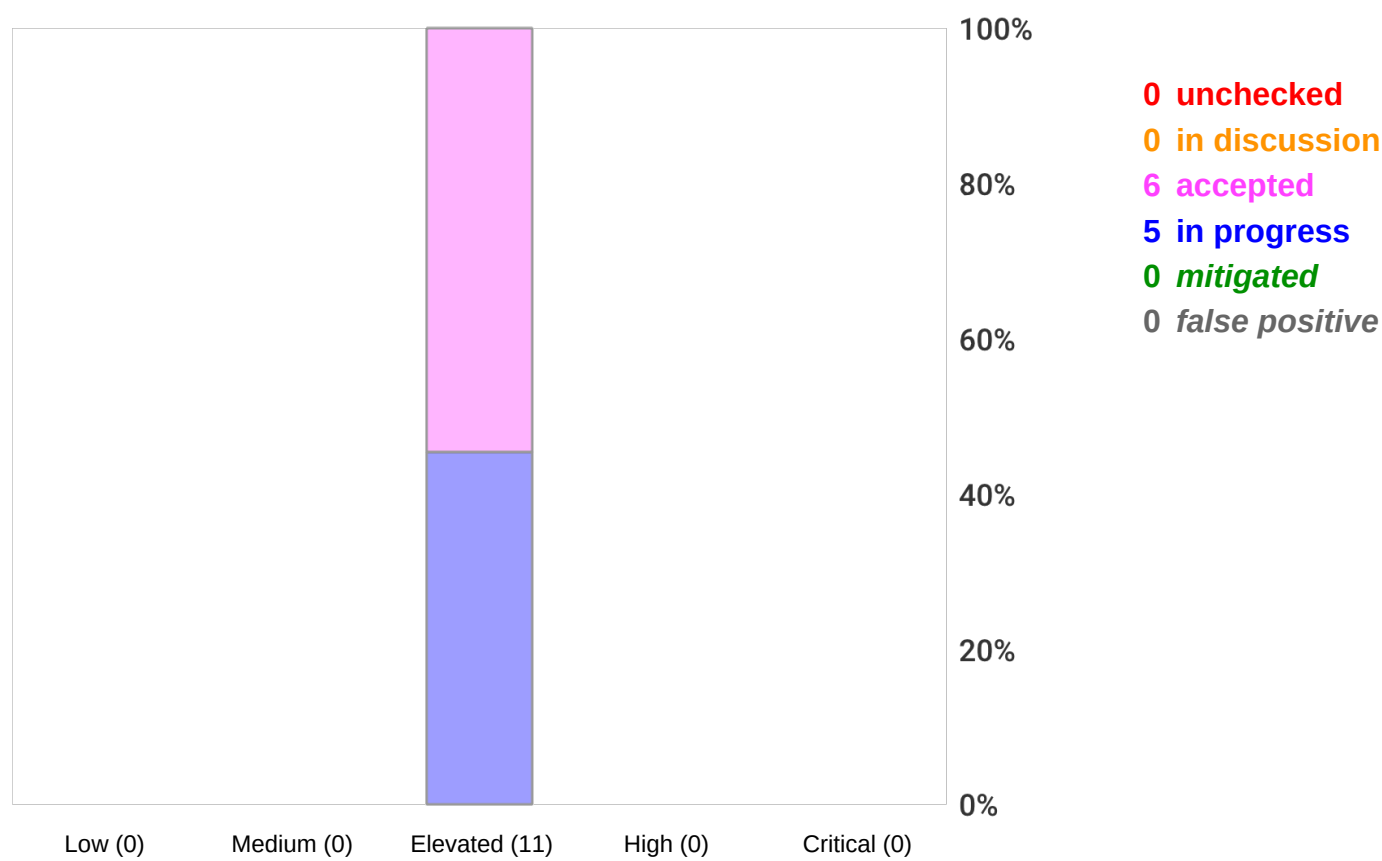
Undetected breaches, inability to investigate incidents, compliance failures, and violation of SLA accountability requirements.

Elevated: **Operational Threat - Critical Business Process on Shared Runtime: 3 Initial Risks** - Exploitation likelihood is *Likely* with *High* impact.

Lateral movement from a compromised co-located workload to the infrastructure supporting a critical business process, causing service outage, data exfiltration, or integrity violations at the process level.

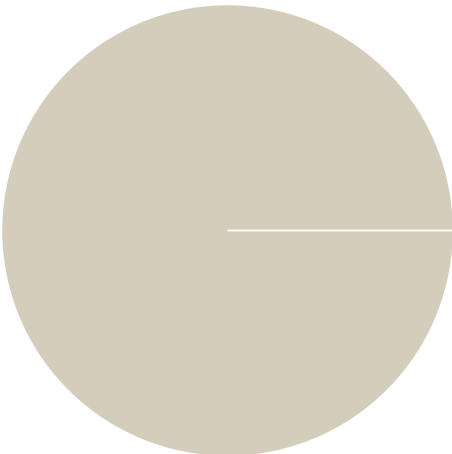
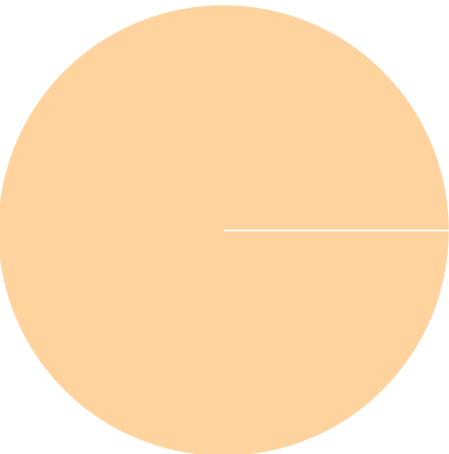
Risk Mitigation

The following chart gives a high-level overview of the risk tracking status (including mitigated risks):



After removal of risks with status *mitigated* and *false positive* the following **11** remain unmitigated:

- 0 unmitigated critical risk
 - 0 unmitigated high risk
 - 11 unmitigated elevated risk
 - 0 unmitigated medium risk
 - 0 unmitigated low risk
- 0 business side related
 - 0 architecture related
 - 0 development related
 - 11 operations related



Asset Register

Technical Assets

API Server

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

Browser SPA

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

MinIO Object Storage

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Nginx Reverse Proxy

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

PostgreSQL Database

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

Redis Cache

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

Data Assets

File Attachments

Binary file uploads attached to notes. May include documents, images, or other sensitive files depending on note usage.

Note Content

User-authored note text — may include personal, financial, or health information depending on how users employ the application.

Session Tokens

JWT tokens and Redis session keys used to maintain authenticated state. Theft allows session hijacking without knowing the user's password.

User Credentials

Email addresses and bcrypt-hashed passwords used for authentication. Compromise allows full account takeover.

Impact Analysis of 11 Remaining Risks in 4 Categories

The most prevalent impacts of the **11 remaining risks** (distributed over **4 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated: Application Threat - Strictly-Confidential Asset Bridges Business Process Contexts: 2 Remaining Risks - Exploitation likelihood is *Likely* with *High* impact.

Credential exposure in any one process context compromises all other processes the asset supports, amplifying the breach blast radius.

Elevated: Operational Threat - Critical Business Process Supported by Non-Redundant Assets: 3 Remaining Risks - Exploitation likelihood is *Likely* with *High* impact.

Complete outage of a critical business process due to failure or targeted attack against the sole supporting technical asset. No failover path means maximum downtime.

Elevated: Operational Threat - Critical Business Process Without Operational Monitoring: 3 Remaining Risks - Exploitation likelihood is *Likely* with *High* impact.

Undetected breaches, inability to investigate incidents, compliance failures, and violation of SLA accountability requirements.

Elevated: Operational Threat - Critical Business Process on Shared Runtime: 3 Remaining Risks - Exploitation likelihood is *Likely* with *High* impact.

Lateral movement from a compromised co-located workload to the infrastructure supporting a critical business process, causing service outage, data exfiltration, or integrity violations at the process level.

Application Overview

Business Criticality

The overall business criticality of "VaultNote Threat Model" was rated as:

(archive | operational | **IMPORTANT** | critical | mission-critical)

Business Overview

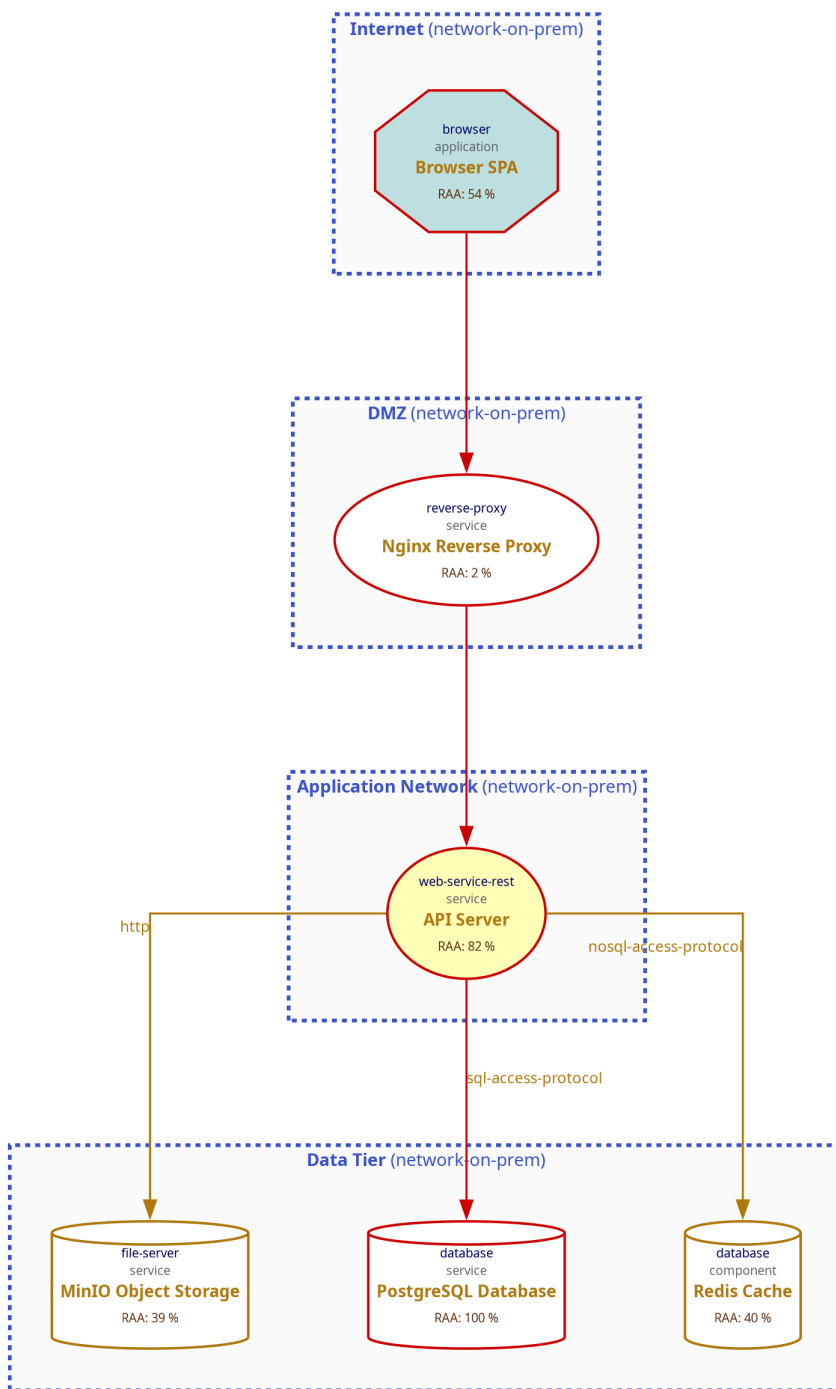
VaultNote allows authenticated users to create, edit, and share encrypted notes with file attachments. The application stores sensitive user data and note content that must be protected against unauthorized disclosure.

Technical Overview

Multi-tier containerized deployment: Nginx terminates TLS and proxies requests to the API over unencrypted HTTP. The API accesses PostgreSQL, Redis, and MinIO over plaintext TCP/HTTP within an internal Docker network.

Data-Flow Diagram

The following diagram was generated by Threagile based on the model input and gives a high-level overview of the data-flow between technical assets. The RAA value is the calculated *Relative Attacker Attractiveness* in percent. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Security Requirements

This chapter lists the custom security requirements which have been defined for the modeled target.

This list is not complete and regulatory or law relevant security requirements have to be taken into account as well. Also custom individual security requirements might exist for the project.

Abuse Cases

This chapter lists the custom abuse cases which have been defined for the modeled target.

This list is not complete and regulatory or law relevant abuse cases have to be taken into account as well. Also custom individual abuse cases might exist for the project.

Tag Listing

This chapter lists what tags are used by which elements.

cache

Redis Cache

container-runtime

Docker Host

docker

Docker Host

express

API Server

intentional-misconfiguration

MinIO File Storage, MinIO Object Storage, HTTP to API Server

jwt

API Server

minio

MinIO Object Storage

nginx

Nginx Reverse Proxy

nodejs

API Server

object-storage

MinIO Object Storage

postgresql

PostgreSQL Database

react

Browser SPA

redis

Redis Cache

relational

PostgreSQL Database

s3

MinIO Object Storage

session-store

Redis Cache

spa

Browser SPA

tls-termination

Nginx Reverse Proxy

STRIDE Classification of Identified Risks

This chapter clusters and classifies the risks by STRIDE categories: In total **11 potential risks** have been identified during the threat modeling process of which **0 in the Spoofing** category, **0 in the Tampering** category, **3 in the Repudiation** category, **2 in the Information Disclosure** category, **3 in the Denial of Service** category, and **3 in the Elevation of Privilege** category.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Spoofing

n/a

Tampering

n/a

Repudiation

Elevated: Operational Threat - Critical Business Process Without Operational Monitoring: 3 / 3 Risks - Exploitation likelihood is *Likely with High impact*.

VAST Operational Threat Model: a business process classified as critical or mission-critical whose supporting technical assets have no communication links with audit logging enabled. Without monitoring, security incidents within this process go undetected, incident response is hindered, and compliance with accountability requirements (GDPR Art. 5(2), SOC 2, ISO 27001 A.12.4) is undermined.

Information Disclosure

Elevated: Application Threat - Strictly-Confidential Asset Bridges Business Process Contexts: 2 / 2 Risks - Exploitation likelihood is *Likely with High impact*.

VAST Application Threat Model: a technical asset that supports business processes and stores or processes strictly-confidential data. Assets that bridge process contexts while holding sensitive credentials or secrets expose those secrets to any threat actor that targets any one of the supported processes, violating the least-privilege boundary between business processes.

Denial of Service

Elevated: Operational Threat - Critical Business Process Supported by Non-Redundant Assets: 3 / 3 Risks - Exploitation likelihood is *Likely with High impact*.

VAST Operational Threat Model: a business process classified as mission-critical or critical that is supported exclusively by technical assets with redundant=false. A single point of failure in the supporting infrastructure can take down the entire critical process, failing service level agreements and potentially causing significant financial or safety impact.

Elevation of Privilege

Elevated: **Operational Threat - Critical Business Process on Shared Runtime: 3 / 3 Risks - Exploitation likelihood is *Likely* with *High* impact.**

VAST Operational Threat Model: a business process marked as mission-critical or critical that depends on a shared runtime (container host, VM cluster) with other, lower-criticality workloads. A compromise of any co-located workload (container escape, host privilege escalation) can directly impact the critical process through shared kernel, storage, or network namespaces.

Assignment by Function

This chapter clusters and assigns the risks by functions which are most likely able to check and mitigate them: In total **11 potential risks** have been identified during the threat modeling process of which **0 should be checked by Business Side**, **0 should be checked by Architecture**, **0 should be checked by Development**, and **11 should be checked by Operations**.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Business Side

n/a

Architecture

n/a

Development

n/a

Operations

Elevated: **Application Threat - Strictly-Confidential Asset Bridges Business Process**

Contexts: 2 / 2 Risks - Exploitation likelihood is *Likely with High* impact.

Issue separate service accounts and credentials scoped to individual process contexts. Use a secrets manager for per-process, short-lived credentials. Consider splitting the asset into process-specific instances.

Elevated: **Operational Threat - Critical Business Process Supported by Non-Redundant Assets: 3 / 3 Risks** - Exploitation likelihood is *Likely with High* impact.

Deploy at least two instances of each technical asset that supports mission-critical or critical business processes. Set redundant to true after adding HA configuration. Implement health checks, automatic failover, and data replication.

Elevated: **Operational Threat - Critical Business Process Without Operational Monitoring: 3 / 3 Risks** - Exploitation likelihood is *Likely with High* impact.

Set audit_logged to true on communication links feeding into technical assets that support critical business processes. Ship logs to a centralised SIEM with alerting on anomalous patterns.

Elevated: **Operational Threat - Critical Business Process on Shared Runtime: 3 / 3 Risks** - Exploitation likelihood is *Likely with High* impact.

Run technical assets supporting critical or mission-critical business processes on dedicated container hosts or hypervisors not shared with untrusted workloads. Use separate node pools, VM sets, or bare-metal where process criticality demands it.

RAA Analysis

For each technical asset the **"Relative Attacker Attractiveness"** (RAA) value was calculated in percent. The higher the RAA, the more interesting it is for an attacker to compromise the asset. The calculation algorithm takes the sensitivity ratings and quantities of stored and processed data into account as well as the communication links of the technical asset. Neighbouring assets to high-value RAA targets might receive an increase in their RAA value when they have a communication link towards that target ("Pivoting-Factor").

The following lists all technical assets sorted by their RAA value from highest (most attacker attractive) to lowest. This list can be used to prioritize on efforts relevant for the most attacker-attractive technical assets:

Technical asset paragraphs are clickable and link to the corresponding chapter.

PostgreSQL Database: RAA 100%

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

API Server: RAA 82%

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

Browser SPA: RAA 54%

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

Redis Cache: RAA 40%

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

MinIO Object Storage: RAA 39%

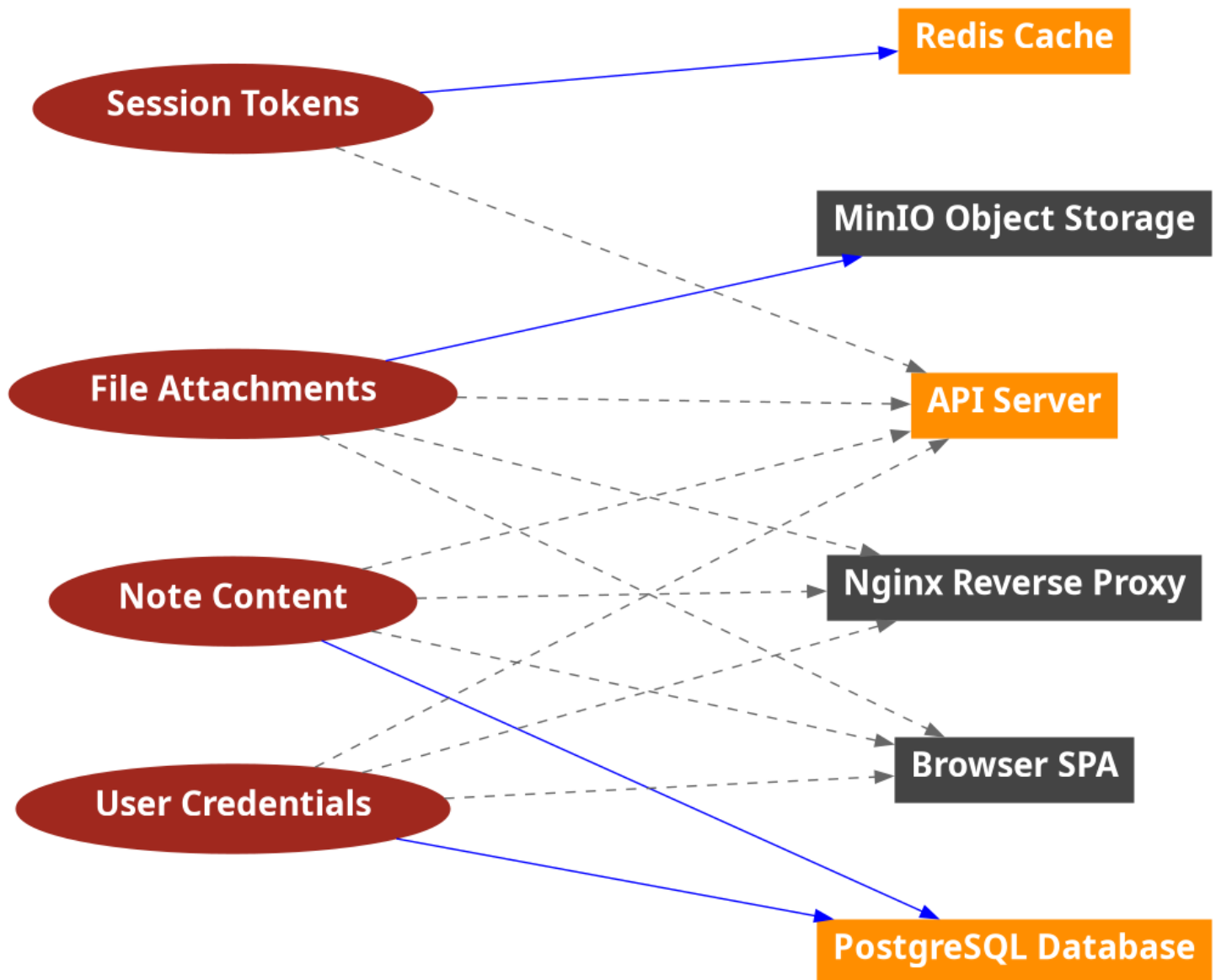
S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Nginx Reverse Proxy: RAA 2%

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Data Mapping

The following diagram was generated by Threagile based on the model input and gives a high-level distribution of data assets across technical assets. The color matches the identified data breach probability and risk level (see the "Data Breach Probabilities" chapter for more details). A solid line stands for *data is stored by the asset* and a dashed one means *data is processed by the asset*. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Out-of-Scope Assets: 0 Assets

This chapter lists all technical assets that have been defined as out-of-scope. Each one should be checked in the model whether it should better be included in the overall risk analysis:

Technical asset paragraphs are clickable and link to the corresponding chapter.

No technical assets have been defined as out-of-scope.

Potential Model Failures: 0 / 0 Risks

This chapter lists potential model failures where not all relevant assets have been modeled or the model might itself contain inconsistencies. Each potential model failure should be checked in the model against the architecture design:

Risk finding paragraphs are clickable and link to the corresponding chapter.

No potential model failures have been identified.

Questions: 0 / 0 Questions

This chapter lists custom questions that arose during the threat modeling process.

No custom questions arose during the threat modeling process.

Identified Risks by Vulnerability category

In total **11 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 11 as elevated, 0 as medium, and 0 as low.**

These risks are distributed across **4 vulnerability categories**. The following sub-chapters of this section describe each identified risk category.

Application Threat - Strictly-Confidential Asset Bridges Business Process

Description (Information Disclosure): [CWE 287](#)

VAST Application Threat Model: a technical asset that supports business processes and stores or processes strictly-confidential data. Assets that bridge process contexts while holding sensitive credentials or secrets expose those secrets to any threat actor that targets any one of the supported processes, violating the least-privilege boundary between business processes.

Impact

Credential exposure in any one process context compromises all other processes the asset supports, amplifying the breach blast radius.

Detection Logic

In-scope technical assets that support at least one business process AND process or store strictly-confidential data.

Risk Rating

High when the asset is internet-accessible; medium for internal-only assets.

False Positives

Shared identity providers (IdPs) architecturally designed to federate identity across processes with explicit isolation controls.

Mitigation (Operations): Isolate strictly-confidential data per business process boundary

Issue separate service accounts and credentials scoped to individual process contexts. Use a secrets manager for per-process, short-lived credentials. Consider splitting the asset into process-specific instances.

ASVS Chapter: [V2 - Authentication Verification Requirements](#)

Cheat Sheet: [Secrets_Management_Cheat_Sheet](#)

Check

Are strictly-confidential data assets and credentials scoped to individual business processes rather than shared via a common technical asset?

Risk Findings

The risk **Application Threat - Strictly-Confidential Asset Bridges Business Process Contexts** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Cross-Process Credential Exposure: API Server bridges business process contexts while handling strictly-confidential data: Exploitation likelihood is *Likely with High* impact.

[application-secrets-across-processes@api-server](#)

[In Progress](#) 2026-04-25 Security Team VAULT-124

The JWT_SECRET environment variable is shared across API server instances for stateless JWT verification. MinIO and PostgreSQL credentials are also shared via compose env vars. Migration to a secrets manager (HashiCorp Vault or AWS Secrets Manager) is planned for production deployment.

Cross-Process Credential Exposure: PostgreSQL Database bridges business process contexts while handling strictly-confidential data: Exploitation likelihood is *Likely with High* impact.

[application-secrets-across-processes@postgresql-db](#)

[In Progress](#) 2026-04-25 Security Team VAULT-124

PostgreSQL POSTGRES_PASSWORD is set via compose environment variable and shared with the API server. Vault-managed dynamic credentials are planned for production.

Operational Threat - Critical Business Process Supported by Non-Redundant Assets

Description (Denial of Service): [CWE 400](#)

VAST Operational Threat Model: a business process classified as mission-critical or critical that is supported exclusively by technical assets with redundant=false. A single point of failure in the supporting infrastructure can take down the entire critical process, failing service level agreements and potentially causing significant financial or safety impact.

Impact

Complete outage of a critical business process due to failure or targeted attack against the sole supporting technical asset. No failover path means maximum downtime.

Detection Logic

Business processes classified as mission-critical or critical where all supporting technical assets have redundant set to false.

Risk Rating

Critical for mission-critical processes; high for critical processes without redundancy.

False Positives

Processes where redundancy is provided at the infrastructure layer (cloud-managed services with implicit HA) that are not modelled as separate assets.

Mitigation (Operations): Add redundancy for all assets supporting critical business processes

Deploy at least two instances of each technical asset that supports mission-critical or critical business processes. Set redundant to true after adding HA configuration. Implement health checks, automatic failover, and data replication.

ASVS Chapter: [V1 - Architecture, Design and Threat Modeling Requirements](#)

Cheat Sheet: [Denial_of_Service_Cheat_Sheet](#)

Check

Are all assets supporting critical or mission-critical business processes deployed with redundancy (redundant=true)?

Risk Findings

The risk **Operational Threat - Critical Business Process Supported by Non-Redundant Assets** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

No Redundancy for Critical Process: API Server supports a critical business process without redundancy: Exploitation likelihood is *Likely with High impact*.

[operational-critical-process-no-redundancy@api-server](#)

Accepted 2026-04-25 Infrastructure Team VAULT-122

API server runs as a single container. Horizontal scaling via docker-compose --scale is tested but not enabled in demo. Load balancer (Nginx upstream pool) will be configured for production.

No Redundancy for Critical Process: PostgreSQL Database supports a critical business process without redundancy: Exploitation likelihood is *Likely with High impact*.

[operational-critical-process-no-redundancy@postgresql-db](#)

Accepted 2026-04-25 Infrastructure Team VAULT-122

Single-instance PostgreSQL is appropriate for the demo environment. Production deployment will use a primary + hot-standby configuration (Patroni). Accepted for demo; tracked as pre-launch requirement.

No Redundancy for Critical Process: Redis Cache supports a critical business process without redundancy: Exploitation likelihood is *Likely with High impact*.

[operational-critical-process-no-redundancy@redis-cache](#)

Accepted 2026-04-25 Infrastructure Team VAULT-122

Redis Sentinel or Redis Cluster is planned for production. For the demo environment, a single Redis instance with AOF persistence is accepted.

Operational Threat - Critical Business Process Without Operational Monitoring

Description (Repudiation): [CWE 778](#)

VAST Operational Threat Model: a business process classified as critical or mission-critical whose supporting technical assets have no communication links with audit logging enabled. Without monitoring, security incidents within this process go undetected, incident response is hindered, and compliance with accountability requirements (GDPR Art. 5(2), SOC 2, ISO 27001 A.12.4) is undermined.

Impact

Undetected breaches, inability to investigate incidents, compliance failures, and violation of SLA accountability requirements.

Detection Logic

Technical assets supporting critical or mission-critical business processes that have no incoming communication link with `audit_logged` set to true.

Risk Rating

High; critical processes without monitoring violate multiple compliance frameworks.

False Positives

Assets monitored by infrastructure-layer logging (cloud-native audit trails) with documented evidence of coverage.

Mitigation (Operations): Enable audit logging on all assets supporting critical business processes

Set `audit_logged` to true on communication links feeding into technical assets that support critical business processes. Ship logs to a centralised SIEM with alerting on anomalous patterns.

ASVS Chapter: [V7 - Error Handling and Logging Verification Requirements](#)

Cheat Sheet: [Logging Cheat Sheet](#)

Check

Do all assets supporting critical business processes have at least one audit-logged communication link?

Risk Findings

The risk **Operational Threat - Critical Business Process Without Operational Monitoring** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Unmonitored Critical Process: API Server supports a critical business process but has no audit-logged ingress: Exploitation likelihood is *Likely with High* impact.

[operational-unmonitored-critical-process@api-server](#)

[In Progress](#) 2026-04-25 Infrastructure Team VAULT-123

Application-level metrics (request rate, error rate, p99 latency) are exposed via /metrics endpoint but Prometheus scraping is not configured in the demo environment. Production monitoring stack is tracked in VAULT-123.

Unmonitored Critical Process: PostgreSQL Database supports a critical business process but has no audit-logged ingress: Exploitation likelihood is *Likely with High* impact.

[operational-unmonitored-critical-process@postgresql-db](#)

[In Progress](#) 2026-04-25 Infrastructure Team VAULT-123

PostgreSQL slow-query log and pg_stat_statements are enabled in the demo config. Full APM integration (Prometheus postgres_exporter + Grafana) is planned for production. Alert thresholds are being defined.

Unmonitored Critical Process: Redis Cache supports a critical business process but has no audit-logged ingress: Exploitation likelihood is *Likely with High* impact.

[operational-unmonitored-critical-process@redis-cache](#)

[In Progress](#) 2026-04-25 Infrastructure Team VAULT-123

Redis INFO metrics are scraped by the Prometheus redis_exporter sidecar in production config but not in the demo compose file. Tracked in VAULT-123.

Operational Threat - Critical Business Process on Shared Runtime: 3 / 3 Risks

Description (Elevation of Privilege): [CWE 284](#)

VAST Operational Threat Model: a business process marked as mission-critical or critical that depends on a shared runtime (container host, VM cluster) with other, lower-criticality workloads. A compromise of any co-located workload (container escape, host privilege escalation) can directly impact the critical process through shared kernel, storage, or network namespaces.

Impact

Lateral movement from a compromised co-located workload to the infrastructure supporting a critical business process, causing service outage, data exfiltration, or integrity violations at the process level.

Detection Logic

Shared runtimes that host assets supporting mission-critical or critical business processes together with assets that do not support any business process.

Risk Rating

High when a critical business process asset shares a runtime with an internet-facing component; elevated otherwise.

False Positives

Dev/test environments where isolation is consciously traded for resource efficiency with documented approval.

Mitigation (Operations): Isolate critical business process workloads onto dedicated runtimes

Run technical assets supporting critical or mission-critical business processes on dedicated container hosts or hypervisors not shared with untrusted workloads. Use separate node pools, VM sets, or bare-metal where process criticality demands it.

ASVS Chapter: [V1 - Architecture, Design and Threat Modeling Requirements](#)

Cheat Sheet: [Docker_Security_Cheat_Sheet](#)

Check

Are technical assets supporting critical business processes isolated from shared runtimes used by

lower-criticality workloads?

Risk Findings

The risk **Operational Threat - Critical Business Process on Shared Runtime** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Shared Runtime - Critical Process: API Server supports a critical business process on a shared runtime: Exploitation likelihood is *Likely with High* impact.

[operational-shared-runtime-critical-process@api-server](#)

Accepted 2026-04-10 Infrastructure Team VAULT-95

See VAULT-95. API server shares the Docker host with datastores. No privileged containers or host mounts beyond named volumes.

Shared Runtime - Critical Process: PostgreSQL Database supports a critical business process on a shared runtime: Exploitation likelihood is *Likely with High* impact.

[operational-shared-runtime-critical-process@postgresql-db](#)

Accepted 2026-04-10 Infrastructure Team VAULT-95

All services share the Docker host as accepted per VAULT-95. No privileged containers; Docker socket not mounted. This is a demo environment constraint.

Shared Runtime - Critical Process: Redis Cache supports a critical business process on a shared runtime: Exploitation likelihood is *Likely with High* impact.

[operational-shared-runtime-critical-process@redis-cache](#)

Accepted 2026-04-10 Infrastructure Team VAULT-95

See VAULT-95. Shared Docker host is accepted for demo; production deployment targets separate VM/node for stateful services.

Identified Risks by Technical Asset

In total **11 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 11 as elevated, 0 as medium, and 0 as low.**

These risks are distributed across **6 in-scope technical assets**. The following sub-chapters of this section describe each identified risk grouped by technical asset. The RAA value of a technical asset is the calculated "Relative Attacker Attractiveness" value in percent.

API Server: 4 / 4 Risks

Description

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

No Redundancy for Critical Process: API Server supports a critical business process without redundancy: Exploitation likelihood is *Likely* with *High* impact.

[operational-critical-process-no-redundancy@api-server](#)

Accepted

2026-04-25

Infrastructure Team

VAULT-122

API server runs as a single container. Horizontal scaling via docker-compose --scale is tested but not enabled in demo. Load balancer (Nginx upstream pool) will be configured for production.

Shared Runtime - Critical Process: API Server supports a critical business process on a shared runtime: Exploitation likelihood is *Likely* with *High* impact.

[operational-shared-runtime-critical-process@api-server](#)

Accepted

2026-04-10

Infrastructure Team

VAULT-95

See VAULT-95. API server shares the Docker host with datastores. No privileged containers or host mounts beyond named volumes.

Cross-Process Credential Exposure: API Server bridges business process contexts while handling strictly-confidential data: Exploitation likelihood is *Likely* with *High* impact.

[application-secrets-across-processes@api-server](#)

In Progress

2026-04-25

Security Team

VAULT-124

The JWT_SECRET environment variable is shared across API server instances for stateless JWT verification. MinIO and PostgreSQL credentials are also shared via compose env vars. Migration to a secrets manager (HashiCorp Vault or AWS Secrets Manager) is planned for production deployment.

Unmonitored Critical Process: API Server supports a critical business process but has no audit-logged ingress: Exploitation likelihood is *Likely* with *High* impact.

[operational-unmonitored-critical-process@api-server](#)

In Progress

2026-04-25

Infrastructure Team

VAULT-123

Application-level metrics (request rate, error rate, p99 latency) are exposed via /metrics endpoint but Prometheus scraping is not configured in the demo environment. Production monitoring stack is tracked in VAULT-123.

Asset Information

ID:	api-server
Type:	process
Usage:	business

RAA:	82 %
Size:	service
Technology:	web-service-rest
Tags:	express, jwt, nodejs
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	File Attachments, Note Content, Session Tokens, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	critical (rated 4 in scale of 5)
CIA-Justification:	The API processes all user data. It bridges both Docker networks — a compromised API container can pivot into the internal data tier.

Outgoing Communication Links: 3

Target technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (outgoing)

Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Target:	Redis Cache
Protocol:	nosql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none

VPN:	false
IP-Filtered:	false
Data Sent:	Session Tokens
Data Received:	Session Tokens

PostgreSQL Connection (outgoing)

Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Target:	PostgreSQL Database
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Note Content, User Credentials
Data Received:	Note Content, User Credentials

MinIO File Storage (outgoing)

INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net can read all stored files.

Target:	MinIO Object Storage
Protocol:	http
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments
Data Received:	File Attachments

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (incoming)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Source:	Nginx Reverse Proxy
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

PostgreSQL Database: 4 / 4 Risks

Description

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

No Redundancy for Critical Process: PostgreSQL Database supports a critical business process without redundancy: Exploitation likelihood is *Likely* with *High* impact.

[operational-critical-process-no-redundancy@postgresql-db](#)

Accepted

2026-04-25

Infrastructure Team

VAULT-122

Single-instance PostgreSQL is appropriate for the demo environment. Production deployment will use a primary + hot-standby configuration (Patroni). Accepted for demo; tracked as pre-launch requirement.

Shared Runtime - Critical Process: PostgreSQL Database supports a critical business process on a shared runtime: Exploitation likelihood is *Likely* with *High* impact.

[operational-shared-runtime-critical-process@postgresql-db](#)

Accepted

2026-04-10

Infrastructure Team

VAULT-95

All services share the Docker host as accepted per VAULT-95. No privileged containers; Docker socket not mounted. This is a demo environment constraint.

Cross-Process Credential Exposure: PostgreSQL Database bridges business process contexts while handling strictly-confidential data: Exploitation likelihood is *Likely* with *High* impact.

[application-secrets-across-processes@postgresql-db](#)

In Progress

2026-04-25

Security Team

VAULT-124

PostgreSQL POSTGRES_PASSWORD is set via compose environment variable and shared with the API server. Vault-managed dynamic credentials are planned for production.

Unmonitored Critical Process: PostgreSQL Database supports a critical business process but has no audit-logged ingress: Exploitation likelihood is *Likely* with *High* impact.

[operational-unmonitored-critical-process@postgresql-db](#)

In Progress

2026-04-25

Infrastructure Team

VAULT-123

PostgreSQL slow-query log and pg_stat_statements are enabled in the demo config. Full APM integration (Prometheus postgres_exporter + Grafana) is planned for production. Alert thresholds are being defined.

Asset Information

ID: postgresql-db
Type: datastore

Usage:	business
RAA:	100 %
Size:	service
Technology:	database
Tags:	postgresql, relational
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Note Content, User Credentials
Data Stored:	Note Content, User Credentials
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	Holds all user PII (email, hashed passwords) and all note content. Strictly confidential — unencrypted disk access exposes all data.	

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

PostgreSQL Connection (incoming)

Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Source:	API Server
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user

Read-Only: false
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Received: Note Content, User Credentials
Data Sent: Note Content, User Credentials

Redis Cache: 3 / 3 Risks

Description

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

No Redundancy for Critical Process: Redis Cache supports a critical business process without redundancy: Exploitation likelihood is *Likely* with *High* impact.

[operational-critical-process-no-redundancy@redis-cache](#)

Accepted

2026-04-25

Infrastructure Team

VAULT-122

Redis Sentinel or Redis Cluster is planned for production. For the demo environment, a single Redis instance with AOF persistence is accepted.

Shared Runtime - Critical Process: Redis Cache supports a critical business process on a shared runtime: Exploitation likelihood is *Likely* with *High* impact.

[operational-shared-runtime-critical-process@redis-cache](#)

Accepted

2026-04-10

Infrastructure Team

VAULT-95

See VAULT-95. Shared Docker host is accepted for demo; production deployment targets separate VM/node for stateful services.

Unmonitored Critical Process: Redis Cache supports a critical business process but has no audit-logged ingress: Exploitation likelihood is *Likely* with *High* impact.

[operational-unmonitored-critical-process@redis-cache](#)

In Progress

2026-04-25

Infrastructure Team

VAULT-123

Redis INFO metrics are scraped by the Prometheus redis_exporter sidecar in production config but not in the demo compose file. Tracked in VAULT-123.

Asset Information

ID:	redis-cache
Type:	datastore
Usage:	business
RAA:	40 %
Size:	component
Technology:	database
Tags:	cache, redis, session-store
Internet:	false
Machine:	container

Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Session Tokens
Data Stored:	Session Tokens
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	important	(rated 3 in scale of 5)
CIA-Justification:	Session token exposure enables account takeover. Non-persistent by default but the container volume could persist RDB snapshots.	

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (incoming)

Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Source:	API Server
Protocol:	nosql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Session Tokens
Data Sent:	Session Tokens

Browser SPA: 0 / 0 Risks

Description

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	browser-spa
Type:	external-entity
Usage:	business
RAA:	54 %
Size:	application
Technology:	browser
Tags:	react, spa
Internet:	true
Machine:	physical
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	true
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	End Users
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Client-side code is public; the trust boundary here is user data entered into the browser. XSS would allow an attacker to exfiltrate session tokens.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (outgoing)

User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Target:	Nginx Reverse Proxy
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	end-user-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

MinIO Object Storage: 0 / 0 Risks

Description

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	minio-storage
Type:	datastore
Usage:	business
RAA:	39 %
Size:	service
Technology:	file-server
Tags:	intentional-misconfiguration, minio, object-storage, s3
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments
Data Stored:	File Attachments
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Stores user file uploads which may contain sensitive documents. Default	

credentials + no encryption = trivial exfiltration by any process on backend-net.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

MinIO File Storage (incoming)

INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net can read all stored files.

Source:	API Server
Protocol:	http
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments
Data Sent:	File Attachments

Nginx Reverse Proxy: 0 / 0 Risks

Description

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	nginx-proxy
Type:	process
Usage:	business
RAA:	2 %
Size:	service
Technology:	reverse-proxy
Tags:	nginx, tls-termination
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	Nginx is the single external entry point. Its compromise exposes all downstream services. Availability is critical — outage = full service outage.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (outgoing)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Target:	API Server
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (incoming)

User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Source:	Browser SPA
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	end-user-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

Identified Data Breach Probabilities by Data Asset

In total **11 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 11 as elevated, 0 as medium, and 0 as low.**

These risks are distributed across **4 data assets**. The following sub-chapters of this section describe the derived data breach probabilities grouped by data asset.

Technical asset names and risk IDs are clickable and link to the corresponding chapter.

File Attachments: 4 / 4 Risks

Binary file uploads attached to notes. May include documents, images, or other sensitive files depending on note usage.

ID:	file-attachments		
Usage:	business		
Quantity:	many		
Tags:	none		
Origin:	customer		
Owner:	VaultNote Team		
Confidentiality:	confidential	(rated 4 in scale of 5)	
Integrity:	critical	(rated 4 in scale of 5)	
Availability:	operational	(rated 2 in scale of 5)	
CIA-Justification:	Files may contain sensitive documents. Integrity is critical because tampered attachments corrupt evidence or business records. MinIO's default credentials (minioadmin) make all stored files accessible to anyone on the network — intentional misconfiguration for demo purposes.		
Processed by:	API Server, Browser SPA, MinIO Object Storage, Nginx Reverse Proxy		
Stored by:	MinIO Object Storage		
Sent via:	MinIO File Storage, HTTPS to Nginx, HTTP to API Server		
Received via:	MinIO File Storage, HTTPS to Nginx, HTTP to API Server		
Data Breach:	probable		
Data Breach Risks:	This data asset has data breach potential because of 4 remaining risksStr:		
	Probable: operational-shared-runtime-critical-process@api-server		
	Probable: application-secrets-across-processes@api-server		
	Improbable: operational-critical-process-no-redundancy@api-server		
	Improbable: operational-unmonitored-critical-process@api-server		

Note Content: 8 / 8 Risks

User-authored note text — may include personal, financial, or health information depending on how users employ the application.

ID:	note-content	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	customer	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Notes may contain highly sensitive personal information. Integrity is critical because corrupted notes lose user trust.	
Processed by:	API Server, Browser SPA, Nginx Reverse Proxy, PostgreSQL Database	
Stored by:	PostgreSQL Database	
Sent via:	PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server	
Received via:	PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 8 remaining risksStr:	

Probable: operational-shared-runtime-critical-process@api-server

Probable: operational-shared-runtime-critical-process@postgresql-db

Probable: application-secrets-across-processes@api-server

Probable: application-secrets-across-processes@postgresql-db

Improbable: operational-critical-process-no-redundancy@api-server

Improbable: operational-critical-process-no-redundancy@postgresql-db

Improbable: operational-unmonitored-critical-process@api-server

Improbable: operational-unmonitored-critical-process@postgresql-db

Session Tokens: 7 / 7 Risks

JWT tokens and Redis session keys used to maintain authenticated state. Theft allows session hijacking without knowing the user's password.

ID:	session-tokens	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Session token compromise enables full session hijacking. Short TTLs and revocation lists mitigate but do not eliminate the risk.	
Processed by:	API Server, Redis Cache	
Stored by:	Redis Cache	
Sent via:	Redis Session Store	
Received via:	Redis Session Store	
Data Breach:	probable	

Data Breach Risks: This data asset has data breach potential because of 7 remaining risksStr:

Probable: operational-shared-runtime-critical-process@api-server

Probable: operational-shared-runtime-critical-process@redis-cache

Probable: application-secrets-across-processes@api-server

Improbable: operational-critical-process-no-redundancy@api-server

Improbable: operational-critical-process-no-redundancy@redis-cache

Improbable: operational-unmonitored-critical-process@api-server

Improbable: operational-unmonitored-critical-process@redis-cache

User Credentials: 8 / 8 Risks

Email addresses and bcrypt-hashed passwords used for authentication. Compromise allows full account takeover.

ID:	user-credentials
Usage:	business
Quantity:	many
Tags:	none
Origin:	customer
Owner:	VaultNote Team
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Credentials grant full account access. A breach leaks all users' identities and enables account takeover across every note and attachment.
Processed by:	API Server, Browser SPA, Nginx Reverse Proxy, PostgreSQL Database
Stored by:	PostgreSQL Database
Sent via:	PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server
Received via:	PostgreSQL Connection

Data Breach: **probable**

Data Breach Risks: This data asset has data breach potential because of 8 remaining risksStr:

Probable: operational-shared-runtime-critical-process@api-server

Probable: operational-shared-runtime-critical-process@postgresql-db

Probable: application-secrets-across-processes@api-server

Probable: application-secrets-across-processes@postgresql-db

Improbable: operational-critical-process-no-redundancy@api-server

Improbable: operational-critical-process-no-redundancy@postgresql-db

Improbable: operational-unmonitored-critical-process@api-server

Improbable: operational-unmonitored-critical-process@postgresql-db

Trust Boundaries

In total **4 trust boundaries** have been modeled during the threat modeling process.

Application Network

Internal Docker bridge network (frontend-net). Contains the API server which is reachable from Nginx but also bridges into backend-net.

ID: application-network
Type: network-on-prem
Tags: none
Assets inside: API Server
Boundaries nested: none

DMZ

Demilitarized zone containing the Nginx reverse proxy. Accessible from the internet but isolated from the application and data tiers.

ID: dmz
Type: network-on-prem
Tags: none
Assets inside: Nginx Reverse Proxy
Boundaries nested: none

Data Tier

Internal Docker bridge network (backend-net, marked internal: true). No external routing. Contains all datastores. The API server has a NIC on this network and acts as the sole legitimate access path.

ID: data-tier
Type: network-on-prem
Tags: none
Assets inside: MinIO Object Storage, PostgreSQL Database, Redis Cache
Boundaries nested: none

Internet

Untrusted external network. All traffic from this zone must cross the Nginx TLS termination point before entering the DMZ.

ID: internet
Type: [network-on-prem](#)
Tags: none
Assets inside: Browser SPA
Boundaries nested: none

Shared Runtimes

In total **1 shared runtime** has been modeled during the threat modeling process.

Docker Host

All containers run on the same Docker host. A container escape or privileged container could access the host filesystem including Docker socket and database volumes.

ID:	docker-host
Tags:	container-runtime, docker
Assets running:	Nginx Reverse Proxy, API Server, PostgreSQL Database, Redis Cache, MinIO Object Storage

Risk Rules Checked by Threagile

Threagile Version: 1.0.0

Threagile Build Timestamp:

Threagile Execution Timestamp: 20260524151526

Model Filename: /home/coder221/Desktop/coding/Threat-model/threagile/threagile.yaml

Model Hash (SHA256): d7bfa142abb73e5d3cd46eac265cb228675b53c92a8d976eddf1abb34b52c1d1

Threagile (see <https://threagile.io> for more details) is an open-source toolkit for agile threat modeling, created by Christian Schneider (<https://christian-schneider.net>): It allows to model an architecture with its assets in an agile fashion as a YAML file directly inside the IDE. Upon execution of the Threagile toolkit all standard risk rules (as well as individual custom rules if present) are checked against the architecture model. At the time the Threagile toolkit was executed on the model input file the following risk rules were checked:

Disclaimer

VaultNote Security Team conducted this threat analysis using the open-source Threagile toolkit on the applications and systems that were modeled as of this report's date. Information security threats are continually changing, with new vulnerabilities discovered on a daily basis, and no application can ever be 100% secure no matter how much threat modeling is conducted. It is recommended to execute threat modeling and also penetration testing on a regular basis (for example yearly) to ensure a high ongoing level of security and constantly check for new attack vectors.

This report cannot and does not protect against personal or business loss as the result of use of the applications or systems described. VaultNote Security Team and the Threagile toolkit offers no warranties, representations or legal certifications concerning the applications or systems it tests. All software includes defects: nothing in this document is intended to represent or warrant that threat modeling was complete and without error, nor does this document represent or warrant that the architecture analyzed is suitable to task, free of other defects than reported, fully compliant with any industry standards, or fully compatible with any operating system, hardware, or other application. Threat modeling tries to analyze the modeled architecture without having access to a real working system and thus cannot and does not test the implementation for defects and vulnerabilities. These kinds of checks would only be possible with a separate code review and penetration test against a working system and not via a threat model.

By using the resulting information you agree that VaultNote Security Team and the Threagile toolkit shall be held harmless in any event.

This report is confidential and intended for internal, confidential use by the client. The recipient is obligated to ensure the highly confidential contents are kept secret. The recipient assumes responsibility for further distribution of this document.

In this particular project, a time box approach was used to define the analysis effort. This means that the author allotted a prearranged amount of time to identify and document threats. Because of this, there is no guarantee that all possible threats and risks are discovered. Furthermore, the analysis applies to a snapshot of the current state of the modeled architecture (based on the architecture information provided by the customer) at the examination time.

Report Distribution

Distribution of this report (in full or in part like diagrams or risk findings) requires that this disclaimer as well as the chapter about the Threagile toolkit and method used is kept intact as part of the distributed report or referenced from the distributed parts.